# Exercise 4: Functions

**Aim**

To implement MySQL functions for banking operations such as calculating a customer's age, computing monthly loan installments, and checking account balance before transactions.

--Utilizing the same database from exercise 1 and 3

--**Question:** Write a function CalculateAge that takes a customer's date of birth as input and returns their age in years.

```
ALTER TABLE Customer

ADD DOB DATE;


UPDATE Customer SET DOB='1961-05-15' WHERE CustomerID=101;

UPDATE Customer SET DOB='1981-09-20' WHERE CustomerID=102;

UPDATE Customer SET DOB='1956-12-10' WHERE CustomerID=103;

UPDATE Customer SET DOB='1991-07-08' WHERE CustomerID=104;

UPDATE Customer SET DOB='1964-03-25' WHERE CustomerID=105;


DELIMITER $$


CREATE FUNCTION CalculateAge(

   DOB DATE

)

RETURNS INT

DETERMINISTIC
```

```
BEGIN

    RETURN TIMESTAMPDIFF(YEAR, DOB, CURDATE());

END $$

DELIMITER ;
SELECT CalculateAge('2004-06-15') AS Age;
```

| | Age |
|---|---|
| ▶ | 22 |

**--Question:** Write a function **CalculateMonthlyInstallment** that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

```
DELIMITER $$

CREATE FUNCTION CalculateMonthlyInstallment(

LoanAmount DECIMAL(10,2),

InterestRate DECIMAL(5,2),

Years INT

)
```

```
RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    DECLARE EMI DECIMAL(10,2);

    SET EMI =
    (LoanAmount +
    (LoanAmount * InterestRate * Years /100))
    /(Years*12);

    RETURN EMI;

END $$

DELIMITER ;

SELECT CalculateMonthlyInstallment(500000,10,5) AS MonthlyInstallment;
```

| | MonthlyInstallment |
|---|---|
| ▶ | 12500.00 |

**--Question:** Write a function **HasSufficientBalance** that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

```
DELIMITER $$

CREATE FUNCTION HasSufficientBalance(
```

```
AccID INT,

Amount DECIMAL(10,2)

)

RETURNS BOOLEAN

DETERMINISTIC

BEGIN

   DECLARE CurrentBalance DECIMAL(10,2);

   SELECT Balance
   INTO CurrentBalance
   FROM Account
   WHERE AccountID = AccID;

   RETURN CurrentBalance >= Amount;

END $$

DELIMITER ;

SELECT HasSufficientBalance(1,5000) AS Status;
```

| Status |
|---|
| 1 |

```
SELECT HasSufficientBalance(1,500000) AS Status;
```

| Status |
|---|
| 0 |